upGrad

#LifeKoKaroLift

SGC Coaching

Advance Session on Neural Networks with TensorFlow|

Optimizers - Intro

upGrad

- Deep Learning, to a large extent, is really about solving massive optimization problems.
- A deep learning model consists of activation function, input, output, hidden layers, loss function, etc.
- While training the deep learning model, we need to modify each epoch's weights and minimize the loss function.
- An optimizer is a function or an algorithm that modifies the attributes of the neural network, such as weights and learning rate.
- Thus, it helps in reducing the overall loss and improve the accuracy.

Why optimizers are needed

upGrad

- The problem of choosing the right weights for the model is a daunting task, as a deep learning model generally consists of millions of parameters.
- It raises the need to choose a suitable optimization algorithm for your application.
- You can use different optimizers to make changes in your weights and learning rate.
- However, choosing the best optimizer depends upon the application.
- As a beginner, one evil thought that comes to mind is that we try all the possibilities and choose the one that shows the best results.
- This might not be a problem initially, but when dealing with hundreds of gigabytes of data, even a single epoch can take a considerable amount of time.
- So randomly choosing an algorithm is no less than gambling with your precious time

how the loss functions and optimizers are related?

upGrad

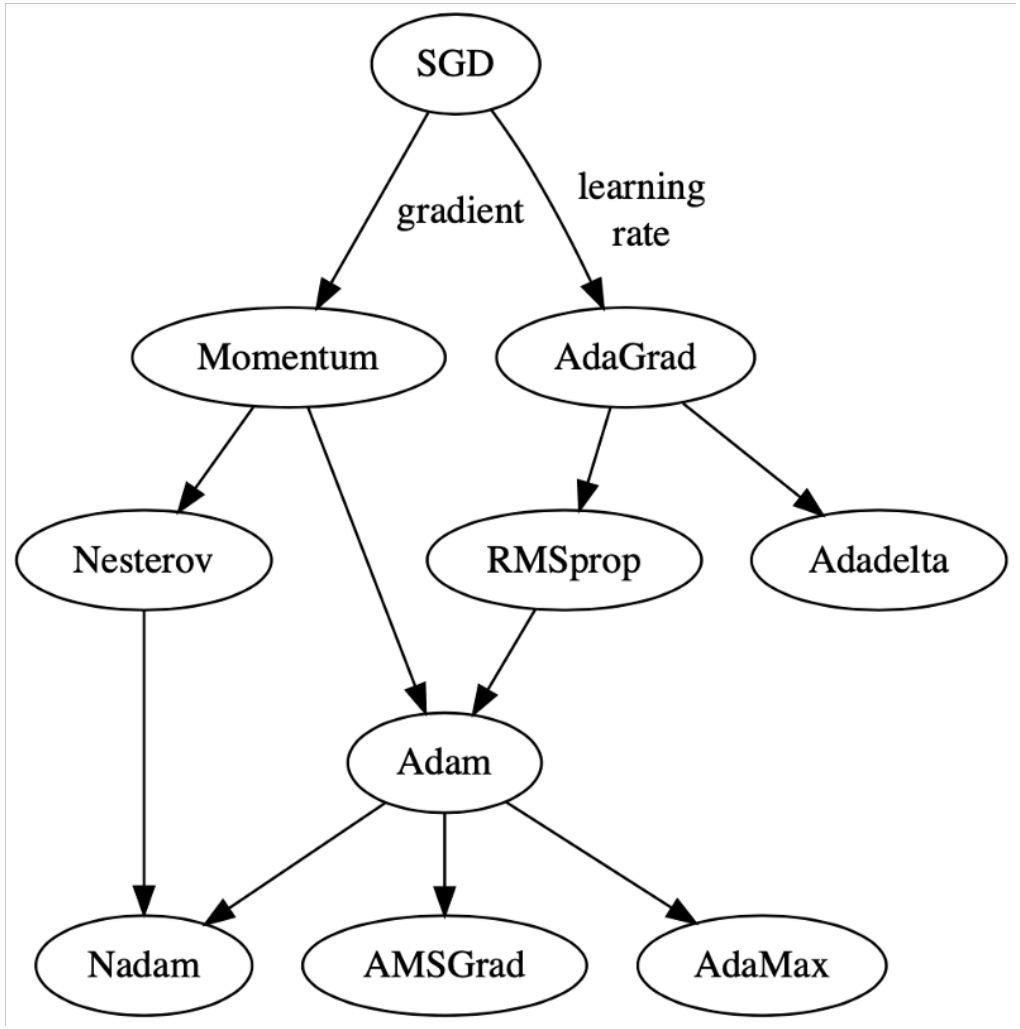
- During the training of the model, we tune the parameters and weights to minimize the loss and try to make our prediction accuracy as correct as possible.
- Now to change these parameters the optimizer's role came in, which ties the model parameters with the loss function by updating the model in response to the loss function output.
- Simply optimizers shape the model into its most accurate form by playing with model weights.
- The loss function just tells the optimizer when it's moving in the right or wrong direction.

Basic terms to be familiar

upGrad

- **Epoch** – The number of times the algorithm runs on the whole training dataset.
- **Sample** – A single row of a dataset.
- **Batch** – It denotes the number of samples to be taken to for updating the model parameters.
- **Learning rate** – It is a parameter that provides the model a scale of how much model weights should be updated.
- **Cost Function/Loss Function** – A cost function is used to calculate the cost that is the difference between the predicted value and the actual value.
- **Weights/ Bias** – The learnable parameters in a model that controls the signal between two neurons

Different types of optimizers


SGD
gradient
learning rate
Momentum
AdaGrad
Nesterov
RMSprop
Adadelta
Adam
Nadam
AMSGrad
AdaMax

- Gradient Descent can be considered as the popular kid among the class of optimizers.
- This optimization algorithm uses calculus to modify the values consistently and to achieve the local minimum.
- **Example**:
- consider you are holding a ball resting at the top of a bowl. When you lose the ball, it goes along the steepest direction and eventually settles at the bottom of the bowl. A Gradient provides the ball in the steepest direction to reach the local minimum that is the bottom of the bowl.

Here alpha is step size that represents how far to move against each gradient with each iteration.

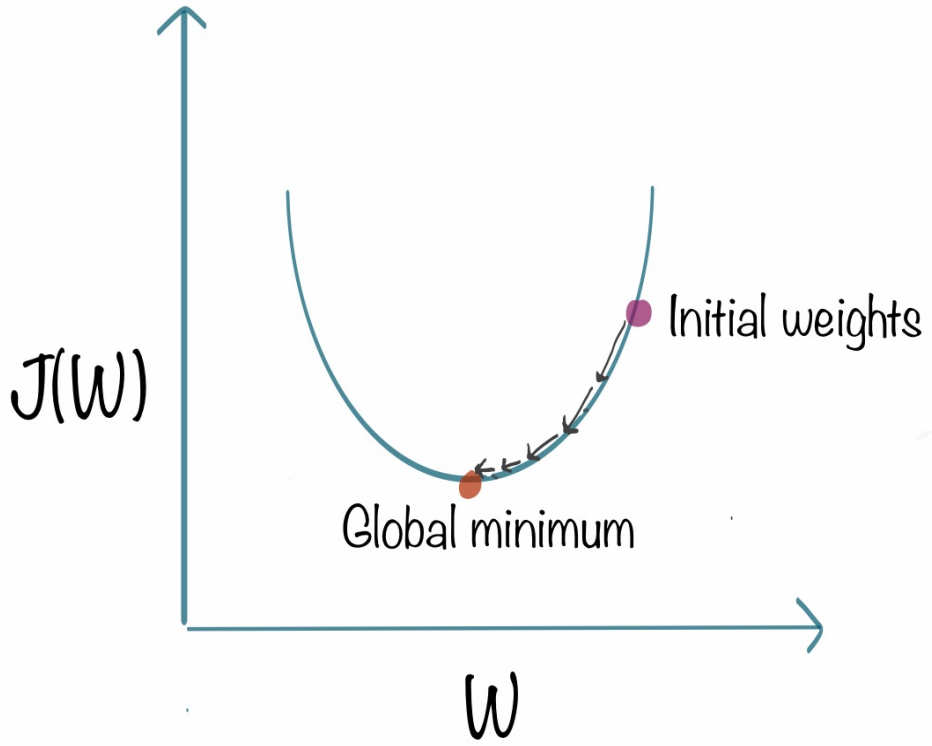
$$x\_new = x - alpha * f'(x)$$

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

upGrad

- It starts with some coefficients, sees their cost, and searches for cost value lesser than what it is now.
- It moves towards the lower weight and updates the value of the coefficients.
- The process repeats until the local minimum is reached.
- A local minimum is a point beyond which it can not proceed.

- Gradient descent works best for most purposes.
- It is expensive to calculate the gradients if the size of the data is huge.
- Gradient descent works well for convex functions but not for nonconvex functions.



Advantages -Disadvantages

upGrad

- **Advantages**:
- Easy computation.
- Easy to implement.
- Easy to understand.

- **Disadvantages**:
- May trap at local minima.
- Weights are changed after calculating gradient on the whole dataset. So, if the dataset is too large than this may take years to converge to the minima.
- Requires large memory to calculate gradient on the whole dataset.

- It’s a variant of Gradient Descent.
- It tries to update the model’s parameters more frequently.
- In this, the model parameters are altered after computation of loss on each training example.
- So, if the dataset contains 1000 rows SGD will update the model parameters 1000 times in one cycle of dataset instead of one time as in Gradient Descent.

- $\theta = \theta - \alpha \cdot \nabla J(\theta; x(i); y(i))$, where {x(i) ,y(i)} are the training examples.

- As the model parameters are frequently updated parameters have high variance and fluctuations in loss functions at different intensities.

- Due to an increase in the number of iterations, the overall computation time increases.
- But even after increasing the number of iterations, the computation cost is still less than that of the gradient descent optimizer.
- So the conclusion is if the data is enormous and computational time is an essential factor, stochastic gradient descent should be preferred over batch gradient descent algorithm.
- To overcome the problem, we use **stochastic gradient descent with a momentum algorithm.**

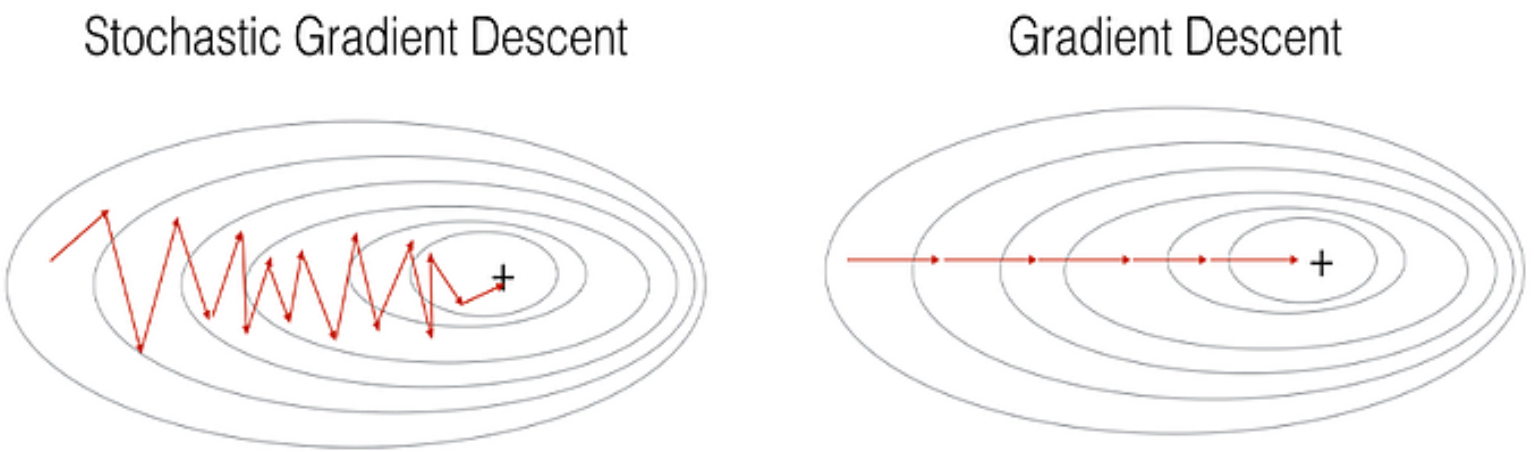


Figure 1 : SGD vs GD

"+" denotes a minimum of the cost. SGD leads to many oscillations to reach convergence. But each step is a lot faster to compute for SGD than for GD, as it uses only one training example (vs. the whole batch for GD).

- **Advantages**:
- Frequent updates of model parameters hence, converges in less time.
- Requires less memory as no need to store values of loss functions.
- May get new minima's.

- **Disadvantages**:
- High variance in model parameters.
- May shoot even after achieving global minima.
- To get the same convergence as gradient descent needs to slowly reduce the value of learning rate.

Momentum

upGrad

- **Momentum**
- Momentum was invented for reducing high variance in SGD and softens the convergence.
- It accelerates the convergence towards the relevant direction and reduces the fluctuation to the irrelevant direction.
- One more hyperparameter is used in this method known as momentum symbolized by '**γ**'.
- $$V(t)=\gamma V(t-1)+\alpha.\nabla J(\theta)$$
- Now, the weights are updated by $\theta=\theta-V(t)$.
- The momentum term **γ** is usually set to 0.9 or a similar value.

- **Advantages**:

- Reduces the oscillations and high variance of the parameters.
- Converges faster than gradient descent.

- **Disadvantages**:

- One more hyper-parameter is added which needs to be selected manually and accurately.

- One of the disadvantages of all the optimizers explained is that the learning rate is constant for all parameters and for each cycle.
- This optimizer changes the learning rate. It changes the learning rate **'η'** for each parameter and at every time step **'t'.**
- It's a type second order optimization algorithm. It works on the derivative of an error function.

$$g_{t,i} = \nabla_\theta J(\theta_{t,i}),$$

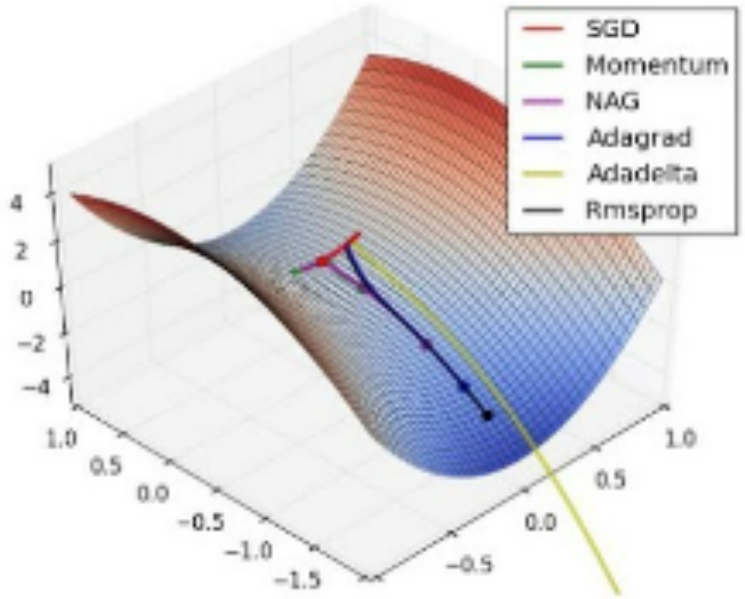
A derivative of loss function for given parameters at a given time t.

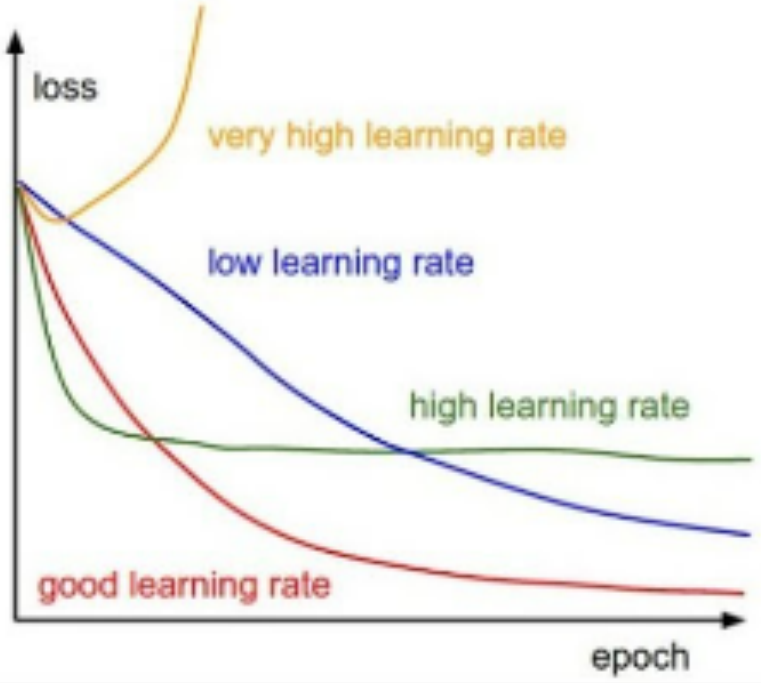
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}.$$

Update parameters for given input i and at time/ iteration t

η is a learning rate which is modified for given parameter **θ(i)** at a given time based on previous gradients calculated for given parameter **θ(i).**

Adagrad Optimizers





It makes big updates for less frequent parameters and a small step for frequent parameters.

Advantages:

- Learning rate changes for each training parameter.
- Don’t need to manually tune the learning rate.
- Able to train on sparse data.

Disadvantages:

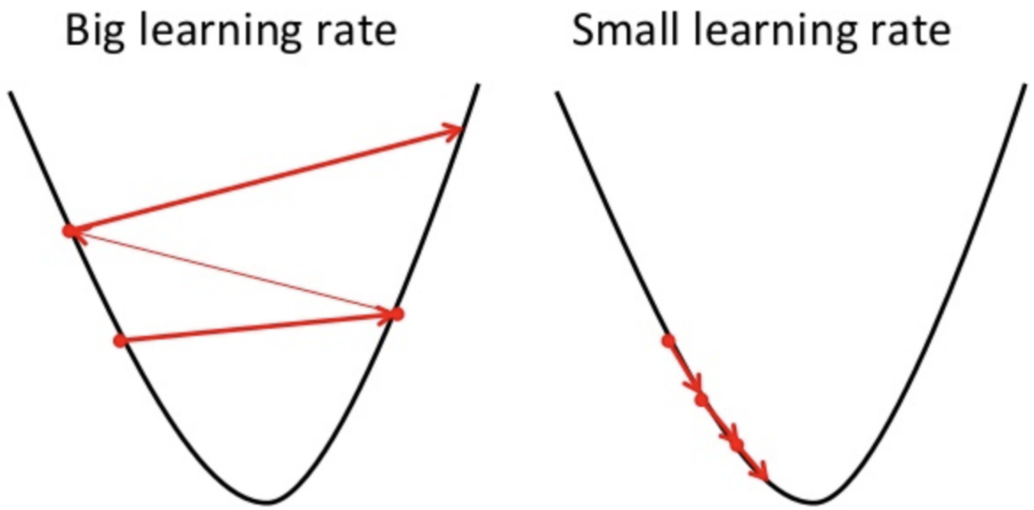
- Computationally expensive as a need to calculate the second order derivative.
- The learning rate is always decreasing results in slow training.

- RMS prop is ideally an extension of the work RPPROP.
- RPPROP resolves the problem of varying gradients.
- RPPROP uses the sign of the gradient adapting the step size individually for each weight.
- In this algorithm, the two gradients are first compared for signs. If they have the same sign, we're going in the right direction and hence increase the step size by a small fraction.
- Whereas, if they have opposite signs, we have to decrease the step size. Then we limit the step size, and now we can go for the weight update.
- Inshort,instead of letting all of the gradients accumulate for momentum, it only accumulates gradients in a specific fix window.
- It is exactly like Adaprop

- The problem with RPPROP is that it doesn't work well with large datasets and when we want to perform mini-batch updates.
- RMS prop can also be considered an advancement in AdaGrad optimizer as it reduces the monotonically decreasing learning rate.
- The algorithm keeps the moving average of squared gradients for every weight and divides the gradient by the square root of the mean square.
- where gamma is the forgetting factor.

$$v(w,t) := \gamma v(w,t-1) + (1-\gamma)(\nabla Q_i(w))^2$$

$$w := w - \frac{\eta}{\sqrt{v(w,t)}} \nabla Q_i(w)$$



- Adam(Adaptive Moment Estimation) works with momentums of first and second order.
- The intuition behind the Adam is that we don't want to roll so fast just because we can jump over the minimum, we want to decrease the velocity a little bit for a careful search.
- In addition to storing an exponentially decaying average of past squared gradients like AdaDelta,
- *Adam* also keeps an exponentially decaying average of past gradients M(t).
- M(t) and V(t) are values of the first moment which is the *Mean* and the second moment which is the *uncentered variance* of the gradients respectively.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

First and second order of momentum

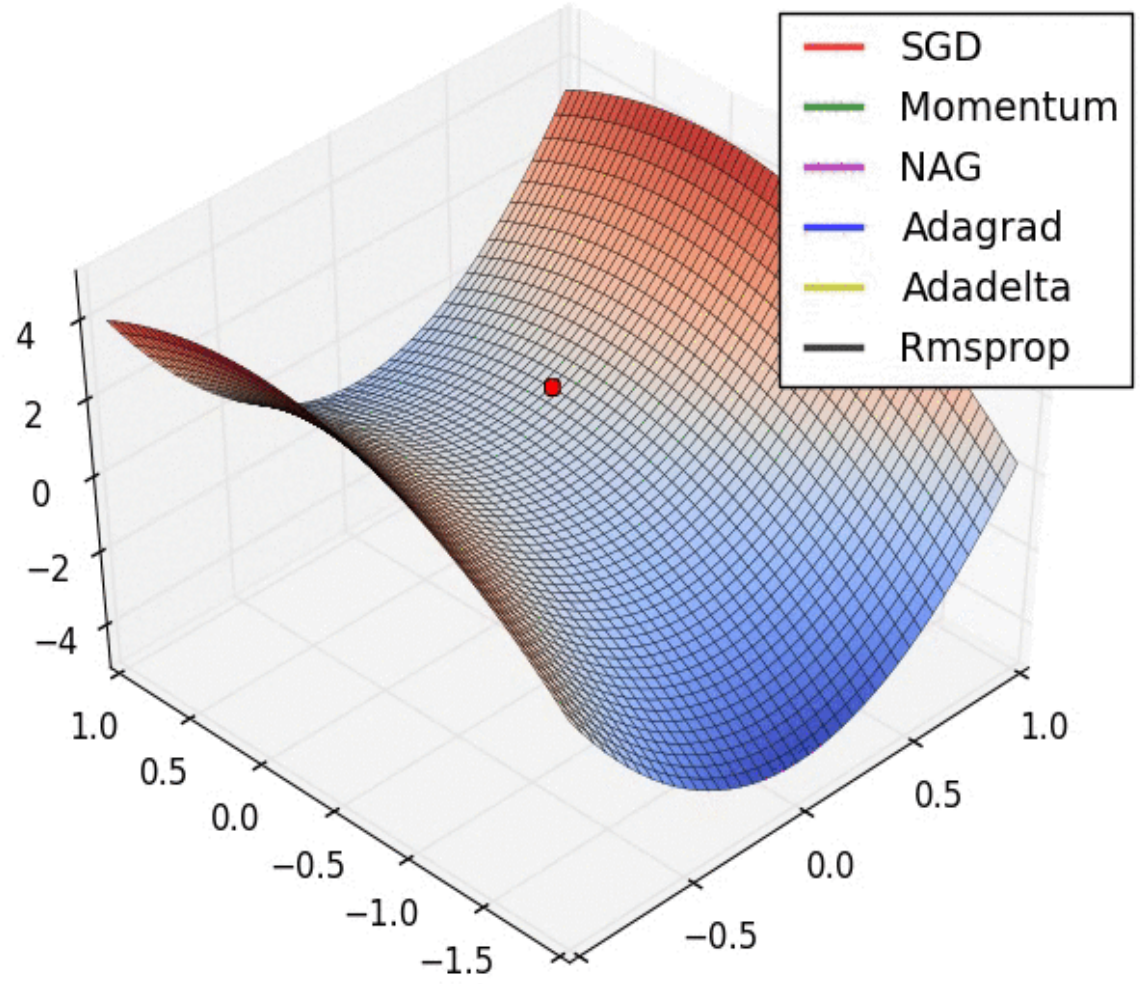
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t.$$

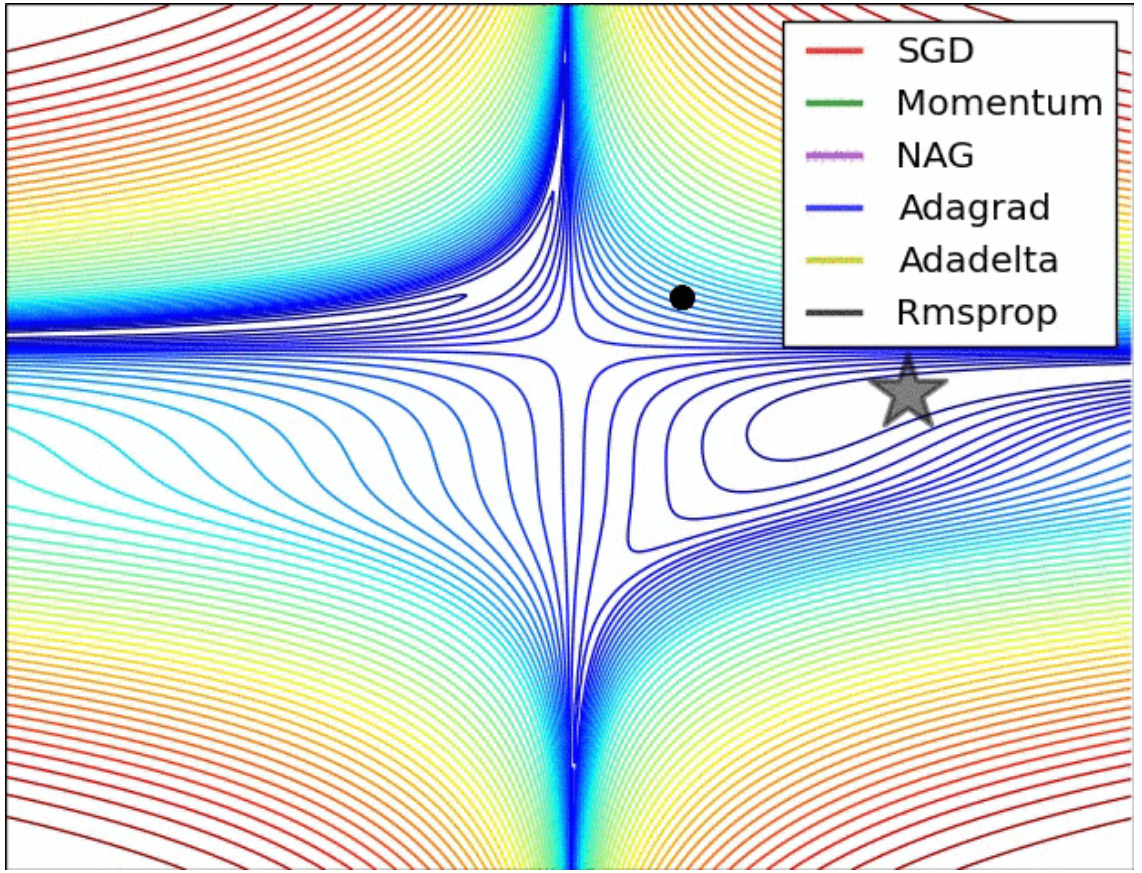
Update the parameters

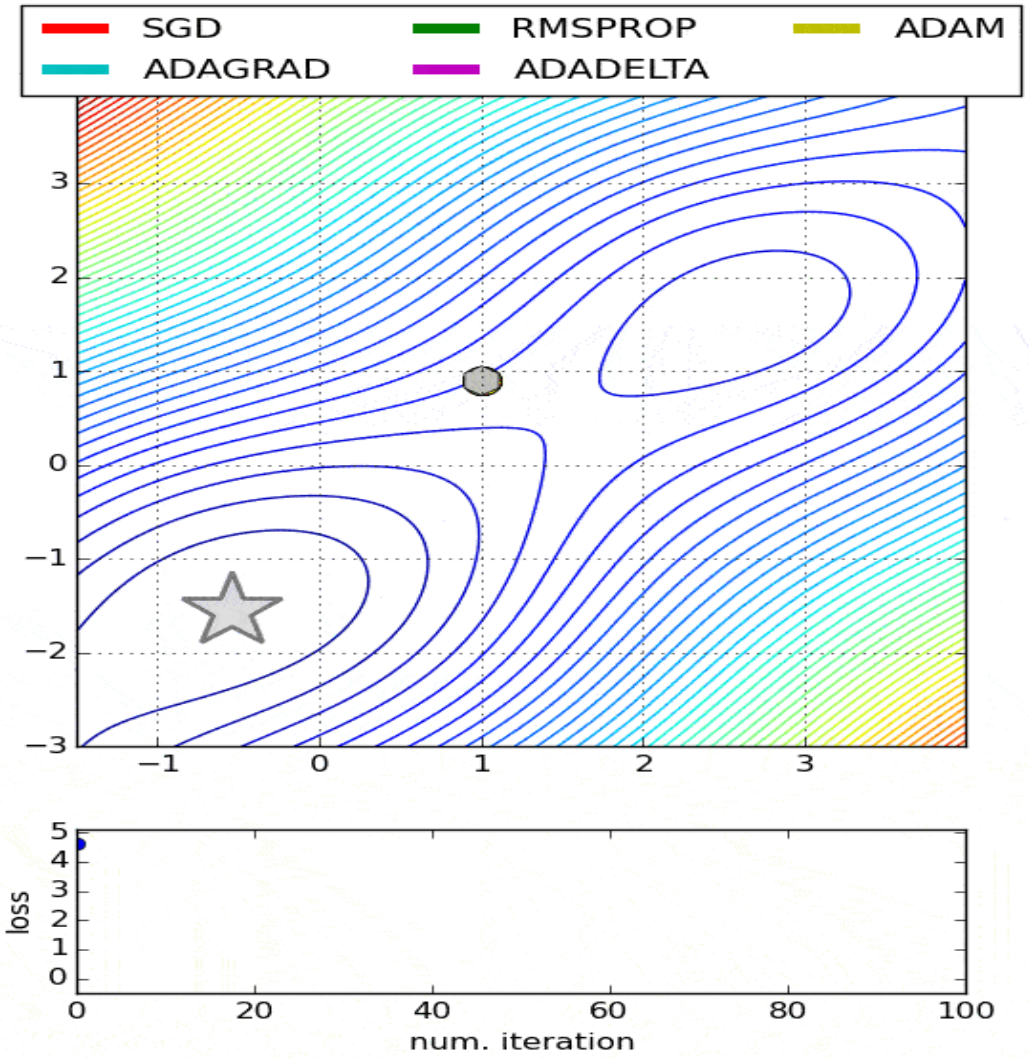
- If the adam optimizer uses the good properties of all the algorithms and is the best available optimizer, then why shouldn’t you use Adam in every application? And what was the need to learn about other algorithms in depth?
- This is because even Adam has some downsides.
- **Advantages**:
- The method is too fast and converges rapidly.
- Rectifies vanishing learning rate, high variance.
- **Disadvantages**:
- Computationally costly.

Comparison between various optimizers

upGrad






SGD
RMSPROP
ADAM
ADAGRAD
ADADELTA
3
2
1
0
−1
−2
−3
−1
0
1
2
3
loss
5
4
3
2
1
0
0
20
40
60
80
100
num. iteration

- Adam is the best optimizers. If one wants to train the neural network in less time and more efficiently than Adam is the optimizer.
- For sparse data use the optimizers with dynamic learning rate.
- If, want to use gradient descent algorithm than min-batch gradient descent is the best option.
- I hope you guys liked the article and were able to give you a good intuition towards the different behaviors of different Optimization Algorithms.